

Concurrency

Definitions

Data Race

- 1. Multiple threads *access* the same memory location.
- 2. At least one access *modifies* the location.
- 3. At least one access is *non-atomic*.
- 4. The accesses are *not ordered* by synchronization.

Data races are *undefined behaviour* in C++.

Race Condition: Multiple interactions between threads that effect execution. Eg: who get’s mut mutex.

Sources of Asynchrony

- Short: Cache Miss
- Medium: Page Fault
- Long: Scheduling Quantum over

Amdahl’s law

Speedup = $\frac{1}{1-p+\frac{p}{n}}$, p =% of time in parallel, n =#threads

Safety and Liveness

- **Safety Property:** Bad things never happen, violatable in finite time.
- **Liveness Property:** Good things eventually happen, only violatable in ∞ time.

C++ Threads, Locks & Condvars

```
namespace std {
class mutex { // not movable/copyable.
    void lock();
    void unlock();
    bool try_lock(); // returns `true` if aquired
}
thread::id this_thread::get_id(); // cur TID.
template <class Mu> scoped_lock {
    // not movable/copyable.
    scoped_lock(Mu mu); // locks
    ~scoped_lock(); // unlocks
}
template <class Mu> unique_lock {
    // movable/copyable, but more expensive
    unique_lock(Mu mu); // locks
    ~unique_lock(); // unlocks
}
class condition_variable {
    condition_variable();
    // 1. Unlock the lock (it should be aquired)
    // 2. Block until woke
    // 3. When woke, reacquire lock.
    void wait(unique_lock<mutex>& lock);
    void wait(unique_lock<mutex>& l, P stop) {uy
        while (!stop()) wait(lock);
    }
    void notify_one();// If threads blocked wake'm
    void notify_all();
} // class condition_variable
class thread() {
    // calls func(args..) on new thread
    thread(func, args...);
    void join(); // must call, UB to leak thread
}
} // namespace std
```

Atomics & Ordering

Atomics allow for well-defined race conditions and fine-grained synchronisation between threads. Map to efficient machine instructions for common types (booleans, integers and pointers).

std::atomic<T>

Defines an atomic variable of type T. Has functions:

- void store(T x): store a value in the atomic variable
- T load(): get the atomic variable’s value
- T exchange(T x): set the atomic’s value to x and return the old value

- **bool compare_exchange_strong(T& expected, T desired):** compare the stored value to *expected and if they are the same, set the value to desired, returning true. Otherwise, store the atomic’s value in *expected and return false. This is all done in one atomic operation.

When T is an integral type, std::atomic<T> also has the following read-modify write operations:

- T **fetch_add**(T x): replace value with the value plus x
- T **fetch_sub**(T x): "" subtract x
- T **fetch_and**(T x): "" bitwise-ANDed with x
- T **fetch_or**(T x): "" bitwise-ORed with x
- T **fetch_xor**(T x): "" bitwise-XORed with x

C++ operator overloading is used so that if x is a std::atomic<int>, ‘x = 4’ has the same effect as ‘x.store(4)’. We *shouldn’t* use these, because it can lead to code with subtle bugs (eg. ‘x += 2’ and ‘x = x + 2’ produce code that behaves differently).

Effect on Data Races

As the 3rd condition of a data race is that at least one access is non-atomic, if all our accesses to a location are atomic, then we can have race conditions, but won’t have data races.

To define how the race conditions behave, we’ll introduce the concept of orderings, and give some examples.

Orderings

We can add an ordering to every **std::atomic** operation. eg. x.**store**(42, std::memory_order_relaxed);

- **Sequential Consistency** (memory_order_seq_cst) The result of an execution can be obtained by some interleaving of every thread where every instruction of each thread appears in program order.
 - Modern multicore CPUs aren’t SC - whilst it’s easy to reason about it’s expensive to implement
 - C++ atomics are sequentially consistent by default

- **Relaxed Memory Order** (memory_order_relaxed) The weakest memory order in C++. The only guarantee is that there is sequential consistency *per location*

- **Release-Acquire** (std::release/std::acquire) Designed for message passing. When an atomic load is done with acquire ordering, bottomthe memory state will be at least as up to date as when the last write to that atomic took place with release ordering. (ie. sender writes with release, then reciever reads with acquire)

Example of where release-acquire has a skill issue:

```
// Thread 0
x.store(1, std::memory_order_release);
// Thread 1
y.store(1, std::memory_order_release);
// Thread 2
print(x.load(std::memory_order_acquire));
print(y.load(std::memory_order_acquire));
// Thread 3
print(y.load(std::memory_order_acquire));
print(x.load(std::memory_order_acquire));
Under release-acquire, T2 could print 1, 0 AND T3 could print 1, 0. Under sequential consistency, this interleaving would not possible.
```

Spinlocks

- Good when expecting to block for a short time: low contention, small critical section, or other domain knowledge
- **Pros:** No syscalls; Simple to implement.

- **Cons:** Busy waiting is slow, lower utilisation from backoff, thrashing, energy use.

TAS Impl

```
class SpinLockSimple {
SpinLockSimple() : is_locked_(false) {}
void Lock() {
    while (is_locked_.exchange(true));
}
void Unlock() { is_locked_.store(false); }
std::atomic<bool> is_locked_;
};
```

Problem: Repeated updates with the same value leads to **cache thrashing**, where caches are invalidated even though data hasn’t changed

Solution: Spin locally (don’t update on every spin, just use .load). Still not great with high contention, as all will observe unlock (when cached value is invalidated) and then attempt to update value.

- **Active Backoff:** after failing to get the lock, wait for some time.
 - An empty for loop may get optimized away, so you want volitile in the loop.
- **Passive backoff:** use intrinsics to tell the CPU your busy waiting (pause in x86, YIELD in ARM). Does not induce context switch.
- **Expontential backoff:** Double the wait time after each attempt to get the lock.

Difficult to get wait time correct - too short and same problems as above; too long and you waste utilisation.

- **Exponential backoff:** Double the wait time after each attempt to get the lock.

```
void Lock() {
    size_t backoff_iterations =
MIN_BACKOFF_ITERATIONS;
    while (is_locked_.exchange(true)) {
        do {
            for (size_t i = 0; i < backoff_iterations; i++)
                _mm_pause();
            backoff_iterations = min(backoff_iterations
* 2, MAX_BACKOFF_ITERATIONS);
        } while (is_locked_.load());
    }
}
```

Ticket Locks: Guarantee fairness.

```
class SpinLockTicket {
    void Lock() {
        size_t ticket = next_ticket_.fetch_add(1);
        while (now_serving_.load() != ticket)
            _mm_pause();
    }
    void Unlock() { now_serving_.fetch_add(1); }
std::atomic<size_t> next_ticket_ = 0;
std::atomic<size_t> now_serving_ = 0;
};
```

Futexes

futex refers to both: *a*) a syscall with SYS_futex (which implements FUTEX_WAIT and FUTEX_WAKE), *b*) user-space mutexes implemented with futex.

Main Futex Operations

- **FUTEX_WAIT(int* p, int v):** If *p == v, sleep on queue associated with p. Otherwise return immediatly.
- **FUTEX_WAKE(int* p, int wake_count):** Wakeup wake_count threads on the queue associated with p. wake_count is usually 1 or INT_MAX.

The int *p can be the address of any int-sized piece of data. Note **sizeof**(std::atomic<int>) == **sizeof(int)** so p can also point to an atomic integer.

Simple futex-based Mutex

```
class Mutex {
    void lock() {
        while(state_.exchange(1) == 1)
            FUTEX_WAIT(&state_, 1);
    }
    void unlock() {
        stat.exchange(0);
        FUTEX_WAKE(&state_, 1);
    }
    std:atomic<int> state_ = 0;
};
```

In fast path, no FUTEX_WAIT syscalls (and therefore no kernel memory) are used. However, even in the uncontended case, a syscall still needed on unlock.

Smarter futex-based Mutex

- We extend the state_ field to store more information.
- 0: Free.
 - 1: Locked, thinks there are no waiters
 - 2: Locked, thinks there are waiters

```
class Mutex {
    void Lock() {
        int e = 0;
        if (state_.compare_exchange(&e, 1))
            return;
        do {
            if (e == 2 || !state_.compare_exchange(&e,
2))
                FUTEX_WAIT(&state_, 2);
            e = 0;
        } while (state_.compare_exchange(&e, 2))
    }
    void Unlock() {
        int old = state_.fetch_sub(1);
        if (old == 2) {
            state_.store(0);
            FUTEX_WAKE(&state_, 1);
        }
    }
    std:atomic<int> state_ = 0;
};
```

Hybrid Locking: We can avoid microcontention by first spinning (say 40 times) and then only sleep (via FUTEX_WAIT) if we still have not managed to acquire the lock.

Vector Clock

- *Threads* = {0, 1, ..., $N - 1$ }: Set of N threads
- *Locks* and *Locations* are sets of shared resources.
- $(c_0, c_1, ..., c_{n-1}) \in VC$ is a Vector Clock. $c_n \in \mathbb{N}$.
- $\perp = (0, 0, ..., 0)$
- $V_1 \sqsubseteq V_2 \Leftrightarrow \forall t. V_1(t) \leq V_2(t)$
- $V_1 \sqcup V_2$: Pointwise maximum.
- $\text{inc}_t(V) = V[t \mapsto V(t) + 1]$

Algorithm State (C, L, R, W)

- $C : Threads \rightarrow VC = (\text{inc}_0 \perp, \text{inc}_1 \perp, ..., \text{inc}_{N-1} \perp)$
- $L : Locks \rightarrow VC = \lambda m. \perp$
- $R : Locations \rightarrow VC = \lambda x. \perp$
- $W : Locations \rightarrow VC = \lambda x. \perp$

Transitions

$$\frac{C' = C[t \mapsto C_t \sqcup L_m]}{(C, L, R, W) \xrightarrow{\text{acquire}(t, m)} (C', L, R, W)}$$
$$\frac{L' = L[m \mapsto C_t] \quad C' = C[t \mapsto \text{inc}_t(C_t)]}{(C, L, R, W) \xrightarrow{\text{release}(t, m)} (C', L', R, W)}$$
$$\frac{W_x \sqsubseteq C_t \quad R' = R[x \mapsto R_x[t \mapsto C_t(t)]]}{(C, L, W, R) \xrightarrow{\text{read}(t, x)} (C, L, W, R')}$$

$$\frac{W_x \sqsubseteq C_t \quad R_x \sqsubseteq C_t \quad W' = W[x \mapsto W_x[t \mapsto C_t(t)]]}{(C, L, W, R) \xrightarrow{\text{write}(t, x)} (C, L, W', R)}$$
$$\frac{\exists u. W_x(u) > C_t(u)}{C, L, W, R \xrightarrow{\text{read}(t, x)} \text{ReadWriteRace}}$$

- u wrote x ; t reads without sync; could be reordered.

$$\frac{\exists u. W_x(u) > C_t(u)}{(C, L, W, R) \xrightarrow{\text{write}(t, x)} \text{WriteWriteRace}}$$

- u wrote x ; t writes without sync; could be reordered.

$$\frac{\exists u. R_x(u) > C_t(u)}{(C, L, W, R) \xrightarrow{\text{write}(t, x)} \text{ReadWriteRace}}$$

- u reads x ; t writes without sync; could be reordered.

Haskell

- **newEmptyMVar :: IO (MVar a)**
- **newMVar :: a -> IO (MVar a):** Contains suplied value
- **takeMVar :: MVar a -> IO a:** Return the contents. If it’s empty, will wait until full. After it’s left empty.
- **putMVar :: MVar a -> a -> IO ():** Put a value into an MVar. If it’s full, wait until it’s empty.
- **readMVar :: MVar a -> IO a:** Read the contents of an MVar. If it’s empty, wait until full.
- **forkIO :: IO () -> IO ThreadId:** Spawns a thread to run the IO action in the first arg.
- **mapM :: Monad m => (a -> m b) -> [a] -> m [b]:** Sequences the monad. Usefull for running a list of actions.

Using MVar to Join threads

```
threadWork :: MVar () -> IO ()
threadWork joinHandle = do
    someConcurrentWork
    putMVar joinHandle () -- Signals completion
```

```
main = do
    handle <- newEmptyMVar
    forkIO (threadWork handle)
    takeMVar handle -- Waits for signal
```

Unbounded Queue

type Stream a = MVar (Item a)
data Item a = Item a (Stream a)

data Channel a
= Channel (MVar (Stream a)) (MVar (Stream a))

```
newChannel :: IO (Channel a)
newChannel = do
    hole <- newEmptyMVar
    readVar <- newMVar hole
    writeVar <- newMVar hole
    return (Channel readVar writeVar)
```

```
readChannel :: Channel a -> IO a
readChannel (Channel readVar _) = do
    stream <- takeMVar readVar
    (Item value tail) <- takeMVar stream
    putMVar readVar tail
    return value
```

```
writeChannel :: Channel a -> a -> IO ()
writeChannel (Channel _ writeVar) value = do
    newHole <- newEmptyMVar
    oldHole <- takeMVar writeVar
    putMVar oldHole (Item value newHole)
    putMVar writeVar newHole
```

Rust

Rust is somewhat like C++, but well-typed programs (without unsafe) are race-free by construction.

Ownership and Borrowing

- Objects have a single owner at any given time.
- Objects are destroyed when their owner goes out of scope

```
fn main() {
  let p = Point{x: 1, y: 2};
  show(p); // main transfers ownership to `show`
  show(p); // ERROR: race after move
}
```

Immutable (Shared) References can be used to allow this. If we say `fn show(point: &Point)` we can call it with `show(&p)`, allowing show to temporarily “borrow” p. Immutable references can alias the underlying object but disallow mutation.

Mutable (Exclusive) references allow modifying borrowed objects:

```
fn move_left(p: &mut Point) { p.x += 1; }
let mut p = Point{x: 1, y: 2};
move_left(&mut p);
```

By enforcing *aliasing XOR mutation*, Rust statically guarantees no data races.

```
let mut p: Point = Point{x: 0, y: 0};
let p_ref: &Point = &p;
// ERROR: cannot borrow `p` as mutable because it is also borrowed as immutable
let p_mut: &mut Point = &mut p;
p_mut.x += 10;
println!("{}", p_ref.x);
```

Threads, Arcs, Mutexs (std types)

```
let t1 = thread::spawn(|| println!("I'm t1"));
let t2 = thread::spawn(|| println!("I'm t2"));
t1.join().unwrap();
t2.join().unwrap();
```

- `|| { ... }` creates a closure with no arguments
- `join()` returns a Result, which can be Ok or Err
- `unwrap()` panics if the result is Err

A **move** closure takes ownership of all captured data.

- Problem:* Threads may outlive the data they reference.
- Solution: Arc<T>* (reference counted pointers)

`arc.clone()` bumps the ref count and returns a new Arc. Ref count is decremented when an Arc goes out of scope. Underlying data is destroyed when `refcount = 0` (i.e. no Arcs left pointing to data).

- Problem:* Unable to share data mutably between threads (mutable references are exclusive to prevent races).

- Solution: Mutex<T>*

`mutex.lock()` locks the mutex and returns a **mutex guard**. The lock guard is an exclusive reference (impls `DerefMut`) to the protected data and so allows mutation until it is dropped and the mutex is unlocked. Note we have obtained a mutable reference to the data through an immutable reference to the mutex - **internal mutability**.

Sequential Consistency

Each thread runs instructions sequentially. Different threads can interleave

Notation

- $x, y, z, \dots \in \text{Loc}$: Shared memory locations
- $a, b, c, \dots \in \text{Reg}$: Private (per thread) registers
- $E, E_1, \dots$: Expressions over ints and regs (no shared mem)

- $a := x$: Read from shared memory
- $x := a$: Write to shared memory
- $a := E$: Compute E into a

ConWhile

$B \in \text{Bool} ::= \dots$ and $E \in \text{Exp} ::= \dots$

$C \in \text{Com} ::= \dots$

$| a := E \mid x := a \mid a := x$
 $| a := \text{CAS}(x, E, E) \mid \text{FAA}(x, E)$
 $| \text{skip} \mid C; C' \mid \text{while } B \text{ do } C$
 $| \text{if } B \text{ then } C \text{ else } C$

SC Configurations

- $\tau \in \text{TID}$
- $P \in \text{Prog} \triangleq \text{Tid} \rightarrow \text{Com}$: PC in each thread.
- $M \in \text{Mem} \triangleq \text{Loc} \rightarrow \text{Val}$: Shared memory.
- $s \in \text{Store} \triangleq \text{Reg} \rightarrow \text{Val}$: One thread's locals.
- $S \in \text{SMap} \triangleq \text{Tid} \rightarrow \text{Store}$: All locals.
- $S(\tau)(b)$ is the register b on thread τ
- A triple (P, S, M) describes program state.

Transition Labels

$l \in \text{Lab}$

- ϵ : Empty, no memory
- (R, x, v) : Read val v from loc x .
- (W, x, v) : Write v to x .
- (U, x, v_o, v_n) : Change x from v_o to v_n (Successful RMW).
- $(U, x, v_o, \perp)$: Failed CAS where x wasn't v_o

$C, s \xrightarrow{l}_c C', s'$ evaluates C with store l , giving a simpler command C' , a new store s' , and emitting transition l .

We assume we have function `eval( $B, s$ )` and `eval( $E, s$ )` that produce $\mathbb{Z}$ and $\mathbb{B}$ respectively. As expressions can't access M , the details aren't important.

Easy Transitions (HA!)

- Sequence Head:**
$$\frac{C_1, s \xrightarrow{l}_c C'_1, s'}{C_1; C_2, s \xrightarrow{l}_c C'_1; C_2, s'}$$

- Sequence Use:**
$$\frac{\text{skip}; C, s \xrightarrow{\epsilon}_c C, s}{\text{eval}(s, B) = \text{true}}$$
- If True:**
$$\frac{\text{if } B \text{ then } C_1 \text{ else } C_2, s \xrightarrow{\epsilon}_c C_1, s}{\text{eval}(s, B) = \text{false}}$$
- If False:**
$$\frac{\text{if } B \text{ then } C_1 \text{ else } C_2, s \xrightarrow{\epsilon}_c C_2, s}{\text{eval}(s, B) = \text{true}}$$

- While:**
$$\frac{\text{while } B \text{ do } C \xrightarrow{\epsilon}_c \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip, } s}{v = \text{eval}(s, E) \quad s' = s[a \mapsto v]}$$
- Assign:**
$$\frac{a := E, s \xrightarrow{\epsilon}_c \text{skip, } s'}{a := E, s \xrightarrow{\epsilon}_c \text{skip, } s'}$$

Command Memory Transitions

- Write & Read & FAA:**

$$\frac{v = s(a)}{(W, x, v)} \quad \frac{s' = s[a \mapsto v]}{(R, x, v)} \\ x := a, s \xrightarrow{c}_c \text{skip, } s \quad a := x, s \xrightarrow{c}_c \text{skip, } s'$$

$$\frac{v_n = v_o + \text{eval}(s, E)}{\text{FAA}(x, E, s) \xrightarrow{(U, x, v_o, v_n)}_c \text{skip, } s}$$

- CAS Succeed & CAS Fail:**

$$\frac{v_o = \text{eval}(E_1, s) \quad v_n = \text{eval}(E_2, e) \quad s' = s[a \mapsto 1]}{a := \text{CAS}(x, E_1, E_2), s \xrightarrow{(U, x, v_o, v_n)}_c \text{skip, } s'}$$

$$\frac{v \neq \text{eval}(E_1, s) \quad s' = s[a \mapsto 0]}{a := \text{CAS}(x, E_1, E_2), s \xrightarrow{(U, x, v, \perp)}_c}$$

Program Transitions

$$\frac{P(\tau) = C \quad S(\tau) = s \quad C, s \xrightarrow{\tau}_c C', s' \quad P' = P[\tau \mapsto C'] \quad S' = S[\tau \mapsto s']}{P, S \xrightarrow{\tau}_p P', S'}$$

SC Storage Transitions

- Read & Write:**

$$\frac{M(x) = v}{\tau: (R, x, v)}_m M \quad \frac{M' = M[x \mapsto v]}{\tau: (R, x, v)}_m M'$$

- Successful & Failed RMW:**

$$\frac{M(x) = v_o \quad M' = M[x \mapsto v_n]}{M \xrightarrow{\tau: (U, x, v_o, v_n)}_m M'} \quad \frac{M(x) = v}{M \xrightarrow{\tau: (U, x, v, \perp)}_m M}$$

Joining Storage and Memory Transitions

$$\frac{P, S \xrightarrow{\tau: \epsilon}_p P', S'}{P, S, M \rightarrow P', S', M'} \quad \frac{P, S \xrightarrow{\tau: l}_p P', S' \quad M \xrightarrow{\tau: l}_m M'}{P, S, M \rightarrow P', S', M'}$$

- $\rightarrow^*$ is the reflexive transitive closure of $\rightarrow$

SC Traces

- $M_0 \triangleq \lambda x. 0$ $s_0 \triangleq \lambda a. 0$ $S_0 \triangleq \lambda S. s_0$
- $P_{\text{skip}} \triangleq \lambda \tau. \text{skip}$
- Given P , it's **initial SC-configuration** is (P, S_0, M_0)
- Given P , it's **SC-outcome** is (S, M) if $P, S_0, M_0 \rightarrow^* P_{\text{skip}}, S, M$

Total Store Ordering (TSO) Weaker memory model than SC - allows to reason about system without perfect cache coherency.

TSO is not deterministic and not confluent.

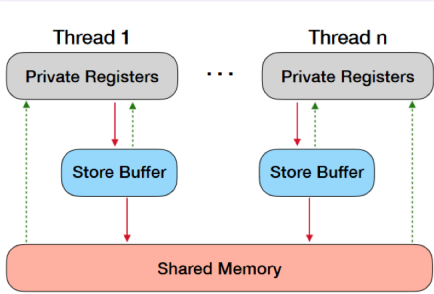
Adjacent instructions can be switched according to the following rules:

- write-read** reordering on *different* locations
- write-to-read** forwarding on the *same* location
- write-assignment** reordering

An **assignment** is a write that doesn't touch memory (just changes local variables).

Write-read ordering directly leads to **store buffering** (where later reads are observed before earlier writes)

Memory fences (mfence in ConWhile) can be placed to avoid reorderings (as instructions are no longer adjacent). mfence is implemented by ensuring the buffer is empty.



We extend the Configuration from SC with buffers: $b \in \text{Buff} \triangleq \text{Seq}(\text{WLab})$

$B \in \text{BMap} \triangleq \text{Tid} \rightarrow \text{Buff}$

Configurations

TSO configuration $\triangleq (P, S, M, B)$

A buffer B is a list of delayed writes, and we all the buffers as part of the configuration.

Program transitions are the same, except for the following mfence rule:

Memory Transitions ($\cdot$ is composition)

- Write & Read:**

$$\frac{B(\tau) = b \quad b' = b.(W, x, v) \quad B' = B[\tau \mapsto b']}{M, B \xrightarrow{\tau: (W, x, v)}_m M, B'}$$

$$\frac{B(\tau) = b \quad \text{get}(M, b, x) = v}{M, B \xrightarrow{\tau: (R, x, v)}_m M, B}$$

$\text{get}(M, b, x) = v$ if $\exists b_1, b_2, b = b_1.(W, x, v).b_2$
 $\text{get}(M, b, x) = M(x)$ otherwise

- MFence & Failed RMW:**

$$\frac{B(\tau) = \emptyset}{M, B \xrightarrow{\text{MF}}_c M, B} \quad \frac{B(\tau) = \emptyset \quad M(x) = v}{M, B \xrightarrow{\tau: (U, x, v, \perp)}_m M, B}$$

- Successful RMW:**

$$\frac{B(\tau) = \emptyset \quad M(x) = v_o \quad M' = M[x \mapsto v_n]}{M, B \xrightarrow{\tau: (U, x, v_o, v_n)}_m M', B}$$

- Flush:**

$$\frac{B(\tau) = (W, x, v).b \quad M' = m[x \mapsto v] \quad B' = B[\tau \mapsto v]}{M, B \xrightarrow{\tau: \epsilon}_m M', B'}$$

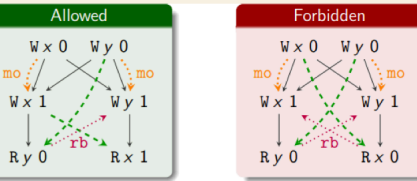
Partial store order (PSO) is weaker than TSO, and allows **write-write** reordering on different locations, and **sfence-read** reordering.

Declarative Semantics Definitions

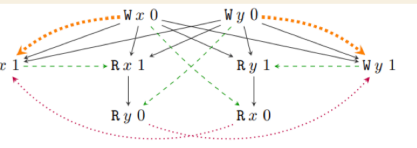
- Event:** A triple $\langle n, \tau, l \rangle$
 - $n \in \mathbb{N}$: a unique identifier
 - $\tau \in \text{Tid} \cup 0$: 0 initializes the values
 - l is non-empty label.
- Label** (we exclude ϵ as we don't need it) is one of:
 - (R, x, v_r)
 - (W, x, v_w)
 - (U, x, v_r, v_w) , where $x \in \text{Loc}$, $v_r, v_w \in \text{Val}$

Graph must be acyclic for relevant memory order to hold. The left graph is SC consistent because there is no cycle (see definitions on formula sheet).

Due to the extra rb edge, there is a cycle in the right graph. The right graph is TSO consistent because the W-R edges aren't in ppo, so there is no cycle.



The below execution is not SC (or TSO) consistent because of a cycle; however, this is the same as the release-acquire example previously, and is RA-consistent.



Preserved Program Order (ppo) is just program order but with the allowed reorderings removed.

Coherence says that SC is observed *per location* $\text{COH} < \text{RA} < \text{TSO} < \text{SC}$ i.e if you can prove COH, you know the others as well

Concurrent Objects

Progress Conditions

	Non-blocking	Blocking
All progress	Wait-free	Starvation-free
One progress	Lock-free	Deadlock-free

- Wait-free $\Rightarrow$ Lock-free
- Starvation-free $\Rightarrow$ Deadlock-free
- Lock-free $\Rightarrow$ Deadlock-free

Sequential Specifications

Can write a sequential specification for an object by specifying every method's precondition, and the postconditions in terms of the method's result and resulting state.

Concurrent Specifications

Method calls are a sequence of intervals (not single events) - i.e. calls can overlap.

We focus on total methods (e.g. returning null or throwing an exception is fine but waiting is problematic) i.e. all blocking should be related to synchronisation.

Linearizable Executions

An object is linearizable if all of it's possible executions are linearizable.

In a linearizable execution, every method should take effect instantaneously, at some point between the invocation and response events. To prove an execution is linearizable, we must identify these “linearization points”.

Notation

- Invocation: `THREAD object.method(args)`
- Response: `THREAD object: ret_val`

Given a history H and method executions m_0 and m_1 : We say $m_0 \rightarrow_H m_1$ if m_0 precedes m_1 in H is a partial order (total if H is sequential)

History

- Matching Events:** An invocation and response are said to “match” if the thread name and object names agree.
- Sequential History:** History is formed entirely by matching event pairs (optionally ending in a pending invocation).
- Well-Formed History:** Every per-thread projection of the history is sequential.
- Equivalent Histories:** The per-thread projections of the history are equal.
- Sequential Specification:** Tells us if a single-thread, single-object history is legal.
- Legal History:** Every per-object projection of the history obeys the sequential specification.

Linearizable History: A history H is linearizable if it can be extended to G by either discarding or appending a response to every pending invocation, such that G is equivalent to a legal sequential history S where $G \xrightarrow{C} S$ (S respects the real-time order of G).

Composability Theorem: A history H is linearizable iff every object projection of H is linearizable.

Sequentially Consistent History: A history H is sequentially consistent if it can be extended to G in the same fashion as linearizability, such that G is equivalent to a legal history S (no need to respect the real-time order - can re-order non-overlapping operations done by different threads).

Sequential consistency is NOT composable.

We can show that an execution fails sequential consistency by illustrating a cycle in *po* $\cup$ *orderings imposed by each object*.